

DEFINICIÓN DE ESTÁNDARES DE CODIFICACIÓN PARA LA PLATAFORMA WEB UKIYO CO

Proyecto Formativo

Construcción de software integrador de tecnologías orientadas a servicios

Programa de Formación: Análisis y Desarrollo de Software

Evidencia: GA7-220501096-AA1-EV01 – Informe técnico de plan de trabajo para construcción de software.

Proyecto de Software: Plataforma Web para la gestión comercial de la marca Ukiyo Co

1. Portada
2. Introducción
3. Descripción del proyecto Ukiyo CO
4. Objetivos
5. Plataforma tecnológica seleccionada
6. Herramientas de desarrollo
7. Herramientas de versionamiento
8. Versionamiento local y remoto
9. Estándares generales de codificación
10. Estándares para HTML5
11. Estándares para CSS3
12. Estándares para JavaScript
13. Estándares para Python
14. Convenciones para nombres
15. Organización de carpetas y archivos
16. Estándares para comentarios y documentación
17. Estándares para Git y GitHub
18. Control de calidad del código
19. Aplicación de los estándares al proyecto Ukiyo CO
20. Conclusiones
21. Referencias

DEFINICIÓN DE ESTÁNDARES DE CODIFICACIÓN PARA LA PLATAFORMA WEB UKIYO CO

1. Introducción

El presente documento establece los estándares de codificación que serán utilizados durante el proceso de construcción de la plataforma web de Ukiyo CO, proyecto orientado al desarrollo de una solución tecnológica para apoyar los procesos de presentación, comercialización y gestión de productos de la marca.

La definición de estándares de codificación permite establecer criterios uniformes para la escritura, organización y mantenimiento del código fuente. Estos criterios contribuyen a mejorar la legibilidad del software, facilitar la identificación de errores y permitir que las diferentes funcionalidades puedan ser modificadas o ampliadas sin afectar de manera innecesaria otros componentes del sistema.

Para el desarrollo de la plataforma se contempla el uso de HTML5, CSS3 y JavaScript como tecnologías principales para el componente Front-End. Asimismo, se contempla Python para el desarrollo de componentes del lado del servidor. Visual Studio Code será utilizado como entorno principal de desarrollo, mientras que Git y GitHub permitirán administrar las diferentes versiones del código.

Los estándares definidos en este documento serán aplicados durante las diferentes etapas de construcción del software, con el propósito de mantener una estructura organizada y establecer criterios comunes para todos los componentes del proyecto.

2. Descripción del proyecto Ukiyo CO

Ukiyo CO es una propuesta de marca de ropa orientada al segmento de moda urbana y streetwear. Como parte del proyecto formativo de Análisis y Desarrollo de Software, se plantea la construcción de una plataforma web que permita fortalecer la presencia digital de la marca y facilitar los procesos relacionados con la presentación y comercialización de sus productos.

La plataforma contará con funcionalidades dirigidas tanto a los clientes como a los administradores.

Entre las funcionalidades destinadas al cliente se encuentran:

- Visualización de la página principal.
- Consulta del catálogo.
- Consulta detallada de productos.
- Visualización de disponibilidad.
- Selección de productos.
- Carrito de compras.
- Registro de usuarios.
- Inicio de sesión.
- Generación de pedidos.
- Comunicación con la marca.

Para el administrador se contemplan funcionalidades como:

- Gestión de productos.
- Gestión de inventario.
- Gestión de usuarios.
- Gestión de pedidos.
- Consulta de información comercial.

La aplicación deberá conservar una identidad visual coherente con la propuesta de Ukiyo CO, utilizando una interfaz basada principalmente en tonos negros y crema, acompañados por un color de acento púrpura.

3. Objetivos

3.1 Objetivo general

Definir los estándares de codificación que serán aplicados durante la construcción de la plataforma web de Ukiyo CO, estableciendo reglas para la organización, escritura, documentación y mantenimiento del código fuente de acuerdo con las tecnologías seleccionadas.

3.2 Objetivos específicos

1. Establecer convenciones para la organización de archivos y carpetas del proyecto.
2. Definir reglas de nomenclatura para variables, funciones, clases, archivos y componentes.
3. Establecer estándares específicos para HTML5, CSS3, JavaScript y Python.
4. Definir criterios para la documentación y utilización de comentarios dentro del código.

5. Establecer reglas para el manejo de versiones mediante Git y GitHub.
6. Promover buenas prácticas que permitan mantener un código legible, organizado y fácil de mantener.

4. Plataforma tecnológica seleccionada

La plataforma tecnológica seleccionada para el desarrollo de Ukiyo CO está compuesta por diferentes tecnologías que cumplen funciones específicas dentro de la arquitectura de la aplicación.

Tecnología	Función
HTML5	Estructura y contenido de las páginas
CSS3	Presentación y diseño visual
JavaScript	Interacción y funcionalidades dinámicas
Python	Desarrollo de componentes del servidor
Git	Control de versiones local
GitHub	Repositorio remoto
Visual Studio Code	Entorno de desarrollo
Figma	Diseño y prototipado
Draw.io	Diagramación del sistema

HTML5 será utilizado para definir la estructura semántica de las páginas. CSS3 permitirá establecer la presentación visual y las características responsive de la interfaz. JavaScript será empleado para gestionar la interacción del usuario y las funcionalidades dinámicas.

Python será utilizado en los componentes que requieran procesamiento del lado del servidor. La selección de estas tecnologías permite separar las responsabilidades de presentación, interacción y procesamiento.

5. Herramientas de desarrollo

5.1 Visual Studio Code

Visual Studio Code será utilizado como entorno principal para la edición del código fuente. La herramienta permitirá trabajar con los archivos HTML, CSS, JavaScript y Python dentro de una estructura organizada.

También facilitará la instalación de extensiones para validación, formato de código, control de versiones y depuración.

5.2 Figma

Figma será utilizado durante la etapa de diseño de interfaces. Los prototipos servirán como referencia para establecer la estructura visual de las diferentes páginas de Ukiyo CO.

5.3 Draw.io

Draw.io será utilizado para representar gráficamente diferentes elementos del sistema, como diagramas de navegación, modelos de datos y estructuras relacionadas con la arquitectura del proyecto.

6. Herramientas de versionamiento

El control de versiones del proyecto estará basado en Git y GitHub.

Git será utilizado de manera local para registrar los cambios realizados sobre el código fuente. Cada modificación significativa deberá registrarse mediante commits descriptivos.

GitHub será utilizado como repositorio remoto para almacenar el proyecto y mantener una copia del historial de modificaciones.

El uso combinado de ambas herramientas permitirá mantener trazabilidad sobre los cambios realizados y facilitar la recuperación de versiones anteriores.

7. Versionamiento local y remoto

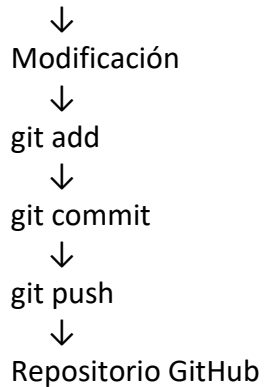
El versionamiento local será administrado mediante Git desde el equipo de desarrollo. El desarrollador podrá realizar modificaciones, crear ramas, registrar commits y consultar el historial del proyecto.

El versionamiento remoto se realizará mediante GitHub. Los cambios registrados localmente podrán enviarse al repositorio remoto utilizando operaciones como git push.

La comunicación entre ambos entornos permitirá mantener sincronizado el proyecto.

El flujo básico será:

Código local



Este procedimiento permitirá mantener un historial de cambios organizado durante el desarrollo.

8. Estándares generales de codificación

Los siguientes estándares serán aplicados de manera transversal en el proyecto:

1. El código deberá mantenerse organizado y estructurado.
2. Los nombres de variables, funciones y archivos deberán ser descriptivos.
3. Se evitará la duplicación innecesaria de código.
4. Las funciones deberán cumplir responsabilidades específicas.
5. Los archivos deberán mantener una extensión correspondiente a la tecnología utilizada.
6. Se utilizará una indentación consistente.
7. Los comentarios deberán utilizarse únicamente cuando aporten información relevante.
8. No se deberán incluir contraseñas, claves privadas o información sensible directamente en el código.
9. Los cambios importantes deberán registrarse mediante Git.
10. Antes de integrar una nueva funcionalidad se deberán realizar pruebas básicas de funcionamiento.
11. Las modificaciones deberán respetar la estructura establecida del proyecto.
12. Los archivos temporales o innecesarios no deberán incluirse en el repositorio.

9. Estándares para HTML5

HTML5 será utilizado para construir la estructura de las páginas web.

Se utilizarán elementos semánticos siempre que sea posible.

Entre ellos:

```
<header>
<nav>
<main>
<section>
<article>
<footer>
```

La utilización de elementos semánticos permitirá mejorar la organización y comprensión del contenido.

Los documentos deberán comenzar con la declaración:

```
<!DOCTYPE html>
```

Se deberá utilizar el atributo lang en el elemento HTML:

```
<html lang="es">
```

Los elementos deberán utilizar una indentación uniforme.

Los atributos deberán utilizar nombres claros y consistentes.

Ejemplo:

```
<section class="catalogo-productos">
  <h2>Productos destacados</h2>

  <article class="producto">
    <h3>Ukiyo Essential Hoodie</h3>
    <p>Descripción del producto.</p>
  </article>
</section>
```

Se deberán utilizar textos alternativos en imágenes mediante el atributo alt para mejorar la accesibilidad.

Ejemplo:

```

```

10. Estándares para CSS3

Los archivos CSS deberán mantenerse separados de los documentos HTML para facilitar la organización y el mantenimiento.

Se utilizarán nombres de clases descriptivos.

Ejemplo:

```
.producto-card {  
    padding: 20px;  
}  
  
.producto-card__titulo {  
    font-size: 20px;  
}  
  
.producto-card__precio {  
    font-weight: bold;  
}
```

Se evitará utilizar nombres de clases genéricos como:

```
.cosa  
.caja  
.texto1  
.estilo2
```

En su lugar, deberán utilizarse nombres relacionados con la función del componente.

También se mantendrá una estructura organizada para los estilos.

Por ejemplo:

1. Variables
2. Estilos generales
3. Navegación
4. Inicio
5. Catálogo
6. Productos
7. Carrito
8. Formularios
9. Panel administrativo
10. Responsive

Los estilos responsive deberán utilizar media queries para adaptar la interfaz a diferentes tamaños de pantalla.

11. Estándares para JavaScript

JavaScript será utilizado para implementar las funcionalidades dinámicas de la plataforma.

Las variables deberán tener nombres descriptivos.

Ejemplo:

```
const nombreProducto = "Buzo Ukiyo";  
const precioProducto = 95000;  
const cantidadProducto = 1;
```

Se deberán utilizar declaraciones modernas como const y let, evitando el uso innecesario de var.

Las funciones deberán tener una responsabilidad claramente definida.

Ejemplo:

```
function calcularTotal(precio, cantidad) {  
    return precio * cantidad;  
}
```

Los nombres de las funciones deberán expresar la acción que realizan.

Ejemplos:

```
agregarProducto()  
eliminarProducto()  
actualizarCarrito()  
validarUsuario()  
calcularTotal()
```

Se evitarán funciones excesivamente extensas y se procurará dividir procesos complejos en funciones más pequeñas.

12. Estándares para Python

En los componentes desarrollados mediante Python se utilizarán nombres descriptivos y una estructura organizada.

Las variables y funciones utilizarán nombres en minúscula separados mediante guiones bajos.

Ejemplo:

```
nombre_producto = "Buzo Ukiyo"  
precio_producto = 95000  
cantidad_producto = 1
```

Las funciones seguirán la misma convención:

```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

Los módulos deberán estar organizados de acuerdo con sus responsabilidades.

Se evitará colocar toda la lógica del sistema en un único archivo.

13. Convenciones para nombres

Se establecerán convenciones diferentes dependiendo de la tecnología.

Elemento	Convención	Ejemplo
Archivo HTML	minúsculas	catalogo.html
Archivo CSS	minúsculas	estilos.css
Archivo JavaScript	minúsculas	carrito.js
Variable JavaScript	camelCase	nombreProducto
Función JavaScript	camelCase	calcularTotal()
Variable Python	snake_case	nombre_producto
Función Python	snake_case	calcular_total()
Clase CSS	kebab-case	producto-card

La utilización de estas convenciones permitirá mantener uniformidad durante el desarrollo.

14. Organización de carpetas y archivos

La estructura propuesta para el proyecto será:

```
UkiyoCO/
├── index.html
├── catalogo.html
├── producto.html
├── carrito.html
├── login.html
├── registro.html
├── admin.html
├── css/
│   ├── estilos.css
│   ├── componentes.css
│   └── responsive.css
├── js/
│   ├── app.js
│   ├── catalogo.js
│   ├── carrito.js
│   ├── login.js
│   └── admin.js
├── img/
│   ├── productos/
│   ├── logos/
│   └── banners/
├── backend/
│   ├── app.py
│   ├── routes/
│   └── models/
├── database/
├── docs/
└── README.md
```

Esta organización permitirá separar los componentes del proyecto de acuerdo con sus funciones.

15. Estándares para comentarios y documentación

Los comentarios deberán utilizarse para explicar aspectos que no sean evidentes a partir del código.

Ejemplo:

```
// Actualiza el total del carrito después de modificar la cantidad.  
function actualizarTotal() {  
    // Procesamiento del total.  
}
```

Se evitarán comentarios innecesarios que simplemente repitan lo que realiza una instrucción.

Incorrecto:

```
// Suma precio y cantidad  
const total = precio * cantidad;
```

Cuando el código sea suficientemente claro, no será necesario añadir comentarios.

Además del código fuente, el proyecto contará con un archivo README.md donde se documentarán aspectos como:

- Descripción del proyecto.
- Requerimientos.
- Tecnologías utilizadas.
- Instalación.
- Ejecución.
- Estructura del proyecto.
- Información sobre versionamiento.

16. Estándares para Git y GitHub

Los commits deberán utilizar mensajes claros que permitan identificar el cambio realizado.

Se propone utilizar las siguientes categorías:

Tipo	Utilización
feat	Nueva funcionalidad
fix	Corrección de error
style	Cambios visuales o de formato

Tipo	Utilización
docs	Modificación de documentación
refactor	Reorganización del código
test	Incorporación o modificación de pruebas

Ejemplos:

feat: agregar catálogo de productos
 feat: implementar carrito de compras
 fix: corregir cálculo del total
 style: ajustar diseño de productos
 docs: actualizar README
 refactor: reorganizar funciones del carrito

Las funcionalidades nuevas deberán desarrollarse preferiblemente en ramas independientes.

Ejemplo:

```
main
|
├── feature/catalogo
├── feature/carrito
├── feature/login
└── feature/inventario
```

Una vez validada la funcionalidad, los cambios podrán integrarse a la rama principal.

17. Control de calidad del código

El cumplimiento de los estándares será revisado durante las diferentes etapas del desarrollo.

La revisión deberá considerar:

- Organización de archivos.
- Nomenclatura.
- Indentación.
- Duplicación de código.
- Funcionamiento de las funcionalidades.
- Errores de sintaxis.

- Adaptabilidad responsive.
- Accesibilidad básica.
- Validación de formularios.
- Funcionamiento de enlaces y navegación.
- Correcto registro de cambios mediante Git.

Antes de incorporar una funcionalidad a la versión estable se deberá comprobar su funcionamiento.

18. Aplicación de los estándares al proyecto Ukiyo CO

Los estándares establecidos se aplicarán directamente a cada uno de los componentes de la plataforma.

En el catálogo de productos se utilizarán nombres descriptivos para los archivos y clases relacionadas con los productos. La información visual estará organizada mediante componentes reutilizables y los estilos se mantendrán separados del contenido HTML.

En el carrito de compras, las funciones JavaScript deberán utilizar nombres que permitan identificar claramente las acciones realizadas, como agregar, eliminar, modificar cantidades y calcular totales.

En el módulo de usuarios se deberán aplicar reglas de validación y organización de datos, evitando almacenar información sensible directamente en el código fuente.

En el panel administrativo se mantendrá una estructura separada para las funcionalidades relacionadas con productos, inventario, usuarios y pedidos.

Todos estos componentes serán administrados mediante Git y deberán registrarse en GitHub conforme avance el desarrollo.

19. Ejemplo de aplicación de los estándares

Un ejemplo de implementación para un producto de Ukiyo CO sería:

```
<article class="producto-card">
  
```

```
<h3 class="producto-card__titulo">
  Buzo Essential
</h3>

<p class="producto-card__precio">
  $95.000
</p>

<button
  class="producto-card__boton"
  id="agregarProducto">
  Agregar al carrito
</button>
</article>
```

La estructura anterior aplica varios de los estándares definidos:

- Utilización de HTML semántico.
- Nombres descriptivos.
- Clases organizadas.
- Uso de atributo alt.
- Separación entre estructura y estilos.
- Identificación clara de los componentes.

La lógica asociada al botón podrá ser desarrollada mediante JavaScript y posteriormente conectada con el sistema de carrito.

20. Beneficios de la implementación de estándares

La aplicación de estándares de codificación proporcionará diferentes beneficios para el proyecto Ukiyo CO.

En primer lugar, facilitará la lectura del código y permitirá identificar rápidamente la función de cada componente. En segundo lugar, contribuirá a reducir errores relacionados con una organización deficiente del proyecto.

Asimismo, permitirá facilitar el mantenimiento y la incorporación de nuevas funcionalidades. Si posteriormente se incorporan métodos de pago, sistemas de notificación, nuevos módulos administrativos o integraciones externas, la estructura establecida permitirá realizar estas modificaciones de manera organizada.

Finalmente, la utilización conjunta de estándares de codificación y control de versiones permitirá mantener un historial de cambios y mejorar la trazabilidad del proceso de desarrollo.

21. Conclusiones

La definición de estándares de codificación constituye un elemento fundamental para garantizar la organización y mantenibilidad de la plataforma web de Ukiyo CO. La implementación de reglas relacionadas con nomenclatura, estructura de archivos, indentación, documentación y organización del código permitirá establecer criterios uniformes durante el proceso de construcción.

La selección de HTML5, CSS3 y JavaScript para el desarrollo del componente Front-End permite separar las responsabilidades relacionadas con estructura, presentación e interacción. De manera complementaria, Python podrá utilizarse para implementar los componentes requeridos del lado del servidor.

El uso de Git y GitHub permitirá controlar las diferentes versiones del proyecto y mantener un registro de los cambios realizados durante la construcción. La utilización de ramas y mensajes descriptivos en los commits facilitará el seguimiento de las funcionalidades desarrolladas.

Finalmente, los estándares definidos en este documento permitirán que el proyecto Ukiyo CO mantenga una estructura técnica organizada y faciliten las futuras actividades de mantenimiento, corrección y ampliación de la plataforma.

22. Referencias

Git. (s. f.). *Git documentation*. <https://git-scm.com/docs>

GitHub. (s. f.). *GitHub documentation*. <https://docs.github.com/>

MDN Web Docs. (s. f.). *HTML: HyperText Markup Language*. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs. (s. f.). *CSS: Cascading Style Sheets*. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (s. f.). *JavaScript*. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Python Software Foundation. (s. f.). *PEP 8 – Style Guide for Python Code*.
<https://peps.python.org/pep-0008/>